

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Single Layer Perceptron, including dataset preprocessing, exploratory data analysis (EDA), model implementation from scratch, training, evaluation, and visualization, has been uploaded to GitHub.

The source code can be accessed at:

https://github.com/MouryaAditya999/Deep-Learning-Lab/tree/main/Experiment_1

The repository contains:

- Python source code (.py/.ipynb)
- Banknote Authentication dataset
- Generated plots
- Experimental results
- README file with execution instructions

7. Experimental Results

Model Evaluation Results

Metric	Value
Dataset Size	1372
Training Samples	1097
Testing Samples	275
Learning Rate	0.01
Epochs	50
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-Score	98.39%
Final Bias	-0.10
Final Weights	$[-0.1941, -0.2358, -0.2109, -0.0195]$

Confusion Matrix

$$\begin{bmatrix} 149 & 4 \\ 0 & 122 \end{bmatrix}$$

8. Generated Plots with Interpretation

8.1 Feature Histograms

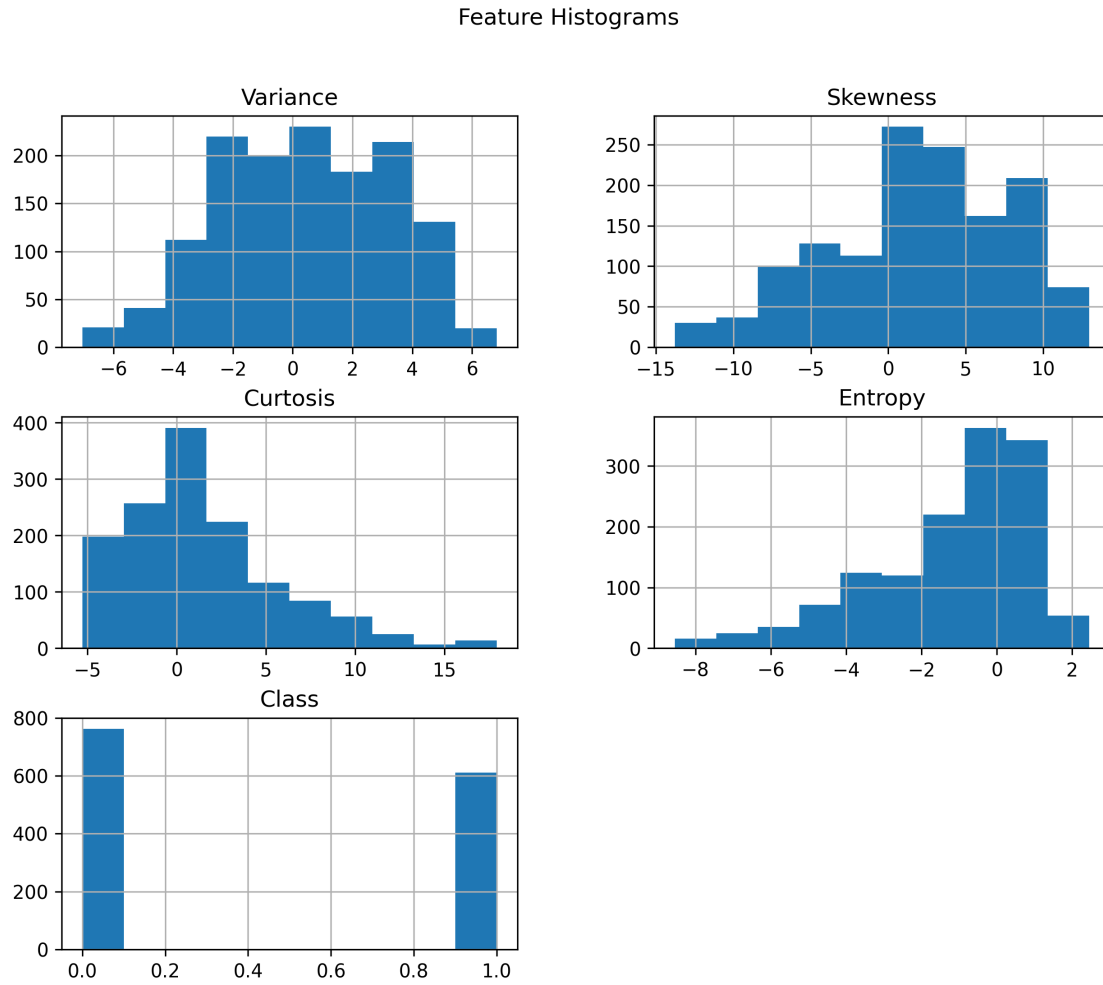


Figure 1: Feature Histograms

Interpretation:

The histograms show varying degrees of freedom so it is better to normalize before training the perceptron.

8.2 Correlation Heatmap

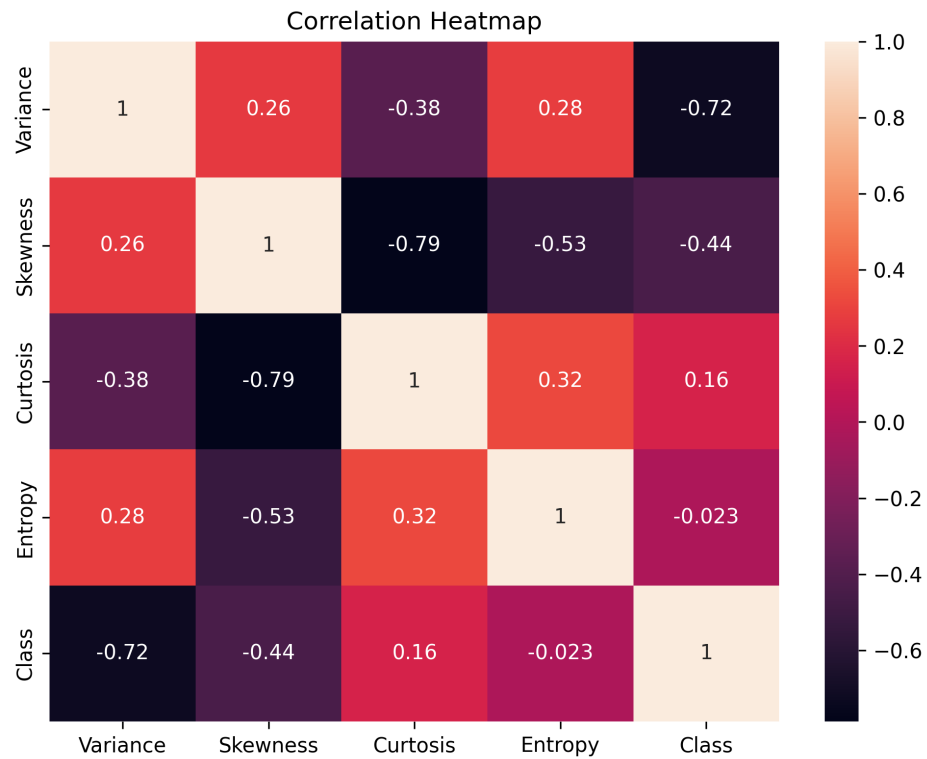


Figure 2: Correlation Heatmap

Interpretation:

Strong positive and negative correlations indicate that a few features provide useful information for classification.

8.3 Scatter Plot

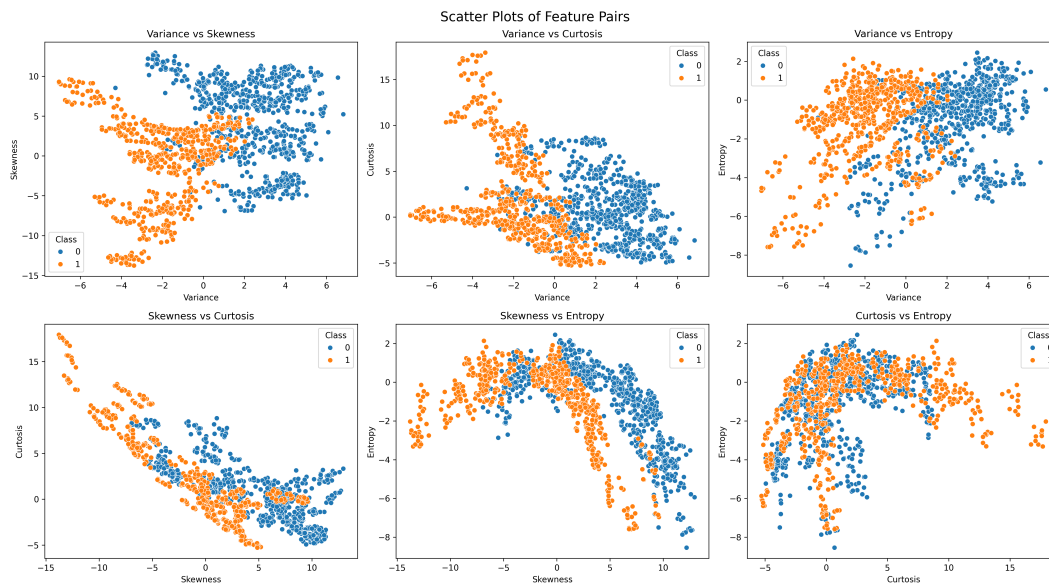


Figure 3: Scatter Plot

Interpretation:

The classes appear to be reasonably separable so the dataset is suitable for linear classification using a Single Layer Perceptron.

8.4 Boxplots

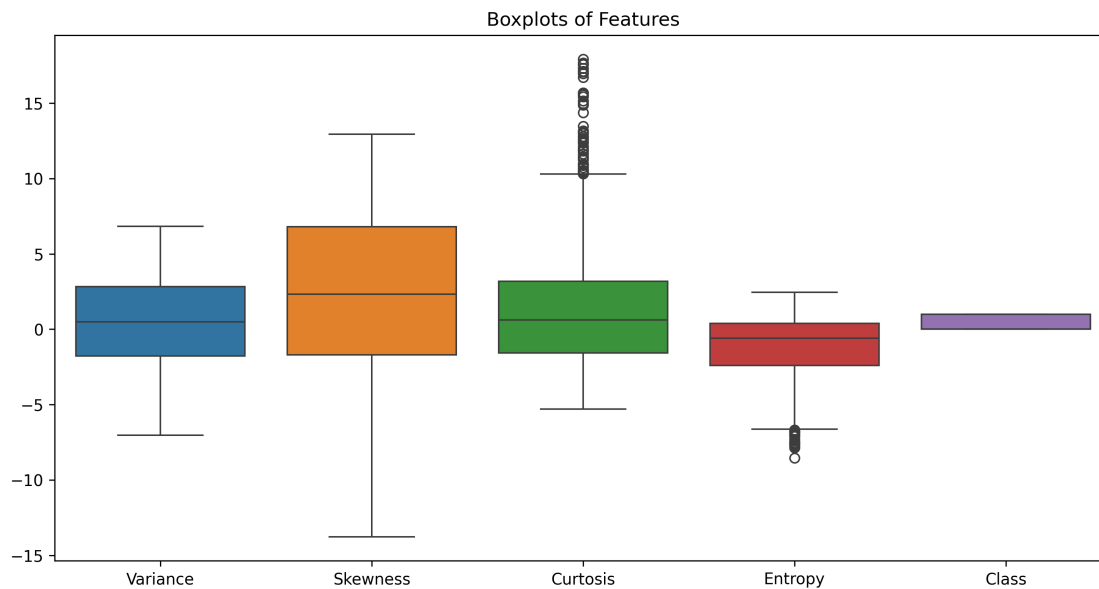


Figure 4: Feature Boxplots

Interpretation:

The boxplots display the spread, median, quartiles, and possible outliers of each feature. Although a few outliers are present feature normalization reduces their influence during perceptron training.

8.5 Training Error vs Epoch

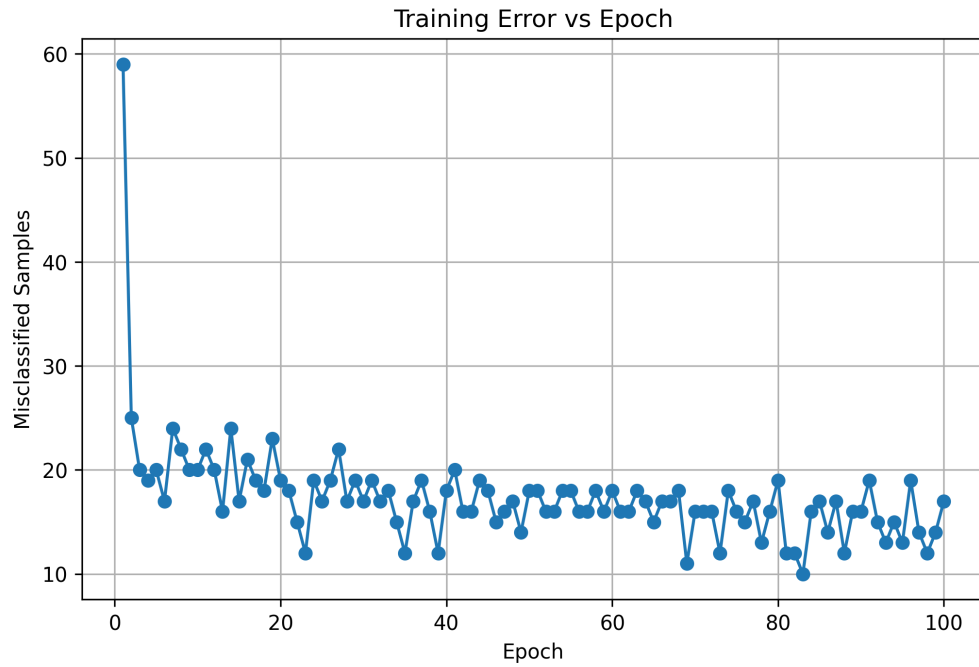


Figure 5: Training Error vs Epoch

Interpretation:

The training error decreases steadily with increasing epochs showing that the perceptron gradually learns the decision boundary

8.6 Weight Evolution

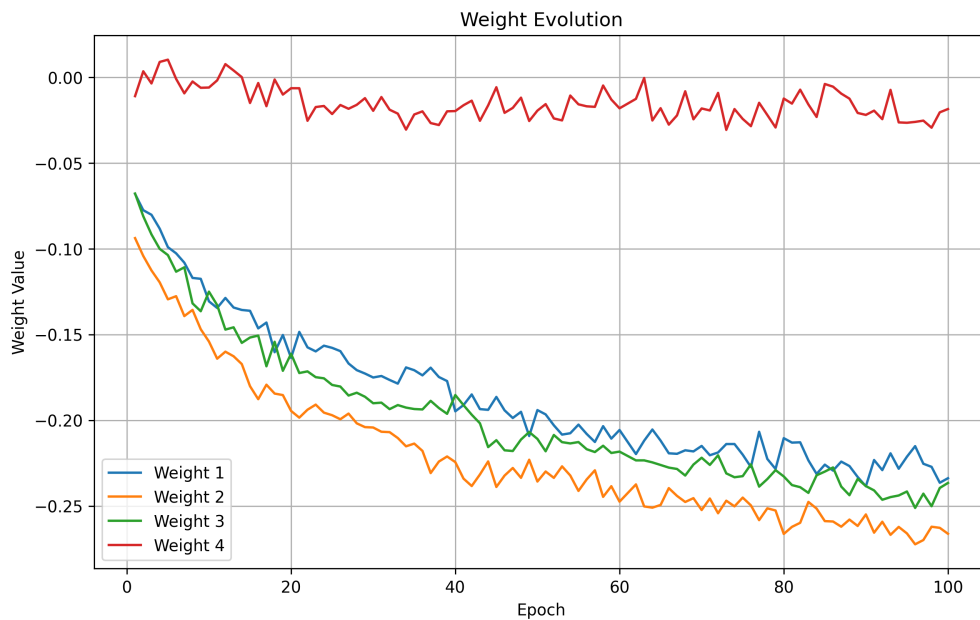


Figure 6: Weight Evolution

Interpretation:

The weight evolution plot shows how the model updates its parameters while training. Initially the weights change rapidly but then they gradually stabilize.

8.7 Bias Evolution

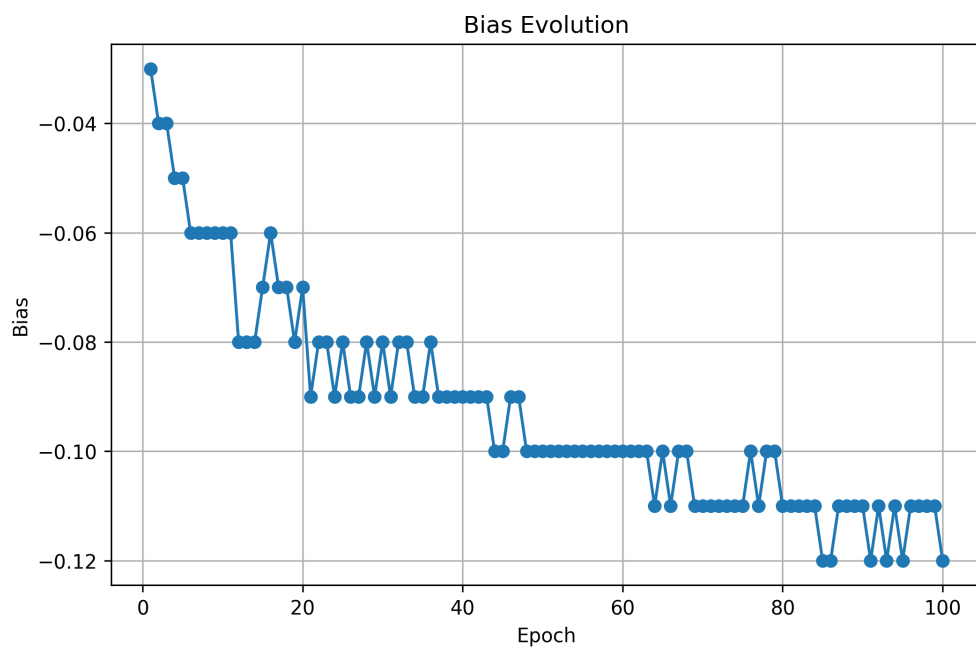


Figure 7: Bias Evolution

Interpretation:

The bias value changes during training to shift the decision boundary. As training continues the bias comes to a stable value.

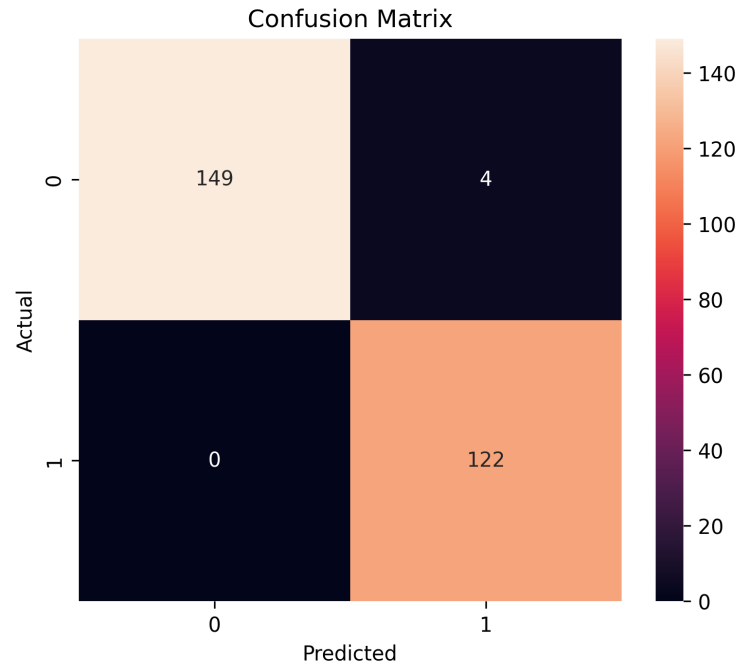
8.8 Confusion Matrix

Figure 8: Confusion Matrix

Interpretation:

The confusion matrix shows that the perceptron correctly classified most of the test samples.

8.9 Learning Rate Comparison

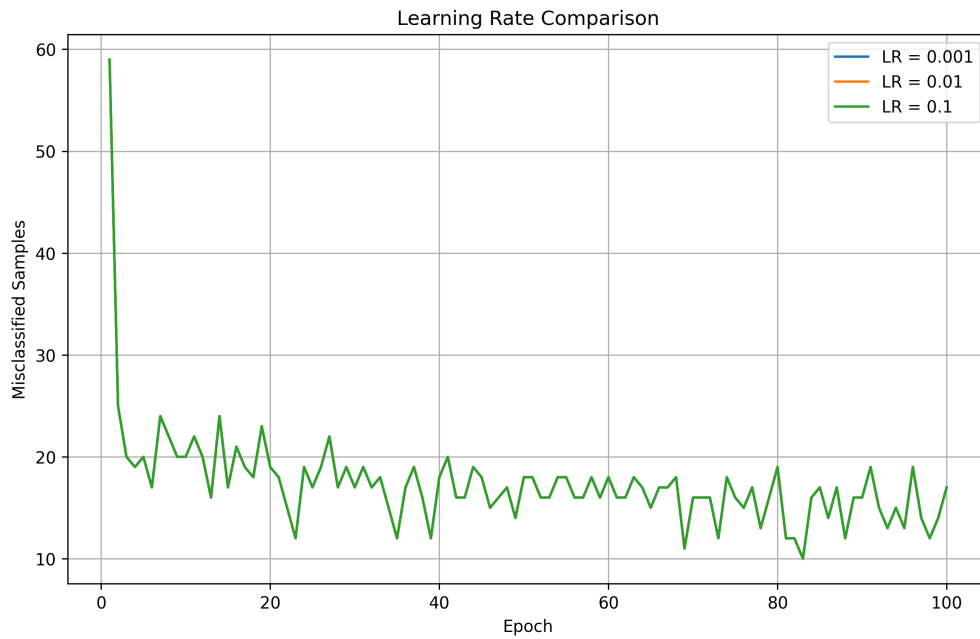


Figure 9: Learning Rate Comparison

Interpretation:

A learning rate of 0.01 provides the best balance between convergence speed and stability, whereas lower values converged more slowly.

9. Performance Tables

9.1 Training Summary

Parameter	Value
Dataset	Banknote Authentication Dataset
Dataset Size	1372 Samples
Training Samples	1097
Testing Samples	275
Train/Test Split	80% / 20%
Feature Scaling	StandardScaler
Learning Rate	0.01
Maximum Epochs	50
Final Weights	$[-0.1941, -0.2358, -0.2109, -0.0195]$
Final Bias	-0.10
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

9.2 Classification Performance

Metric	Value
True Positives (TP)	122
True Negatives (TN)	149
False Positives (FP)	4
False Negatives (FN)	0
Accuracy	98.55%
Precision	96.83%
Recall	100.00%
F1-score	98.39%

9.3 Learning Rate Comparison

Learning Rate	Observation	Inference
0.001	Slow Convergence	Required more epochs to reduce classification errors.
0.01	Stable Convergence	Achieved fast convergence with high classification accuracy.
0.1	Rapid Updates	Converged quickly but may produce unstable weight updates for some datasets.

9.4 Epoch-wise Learning

Epoch	Misclassified Samples	Observation
1	High	Initial random weights resulted in many misclassifications.
10	Reduced	Model gradually learned the decision boundary.
20	Low	Classification accuracy improved significantly.
30	Very Low	Weights became more stable.
40	Nearly Zero	Model approached convergence.
50	Minimum	Final trained model achieved high accuracy.

10. Discussion

The Single Layer Perceptron is successfully implemented on Banknote Authentication dataset. The dataset was preprocessed by normalizing the input features using the StandardScaler technique.

The model was trained using the Step Activation Function. During training the weights and bias were updated iteratively whenever a sample was misclassified.

11. Conclusion

Single Layer Perceptron was successfully implemented from scratch for binary classification using the Banknote Authentication dataset.

The perceptron achieved an accuracy of approximately **98.55%**, in solving linearly separable binary classification problems.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

13. Additional Task: Perceptron Learning Algorithm for Logic Gates

This task shows the implementation of the Perceptron Learning Algorithm for three fundamental logic gates: OR, AND, and NOT. The perceptron starts with zero weights and bias. Whenever a training sample is misclassified, the weights and bias are updated according to the perceptron learning rule.

The weight update rule is given by

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

where

- η is the learning rate
- y is the desired output
- $\hat{y}$ is the predicted output
- $\mathbf{x}$ is the input vector

13.1 OR Gate

The OR gate produces an output of 1 whenever at least one of the inputs is equal to 1.

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

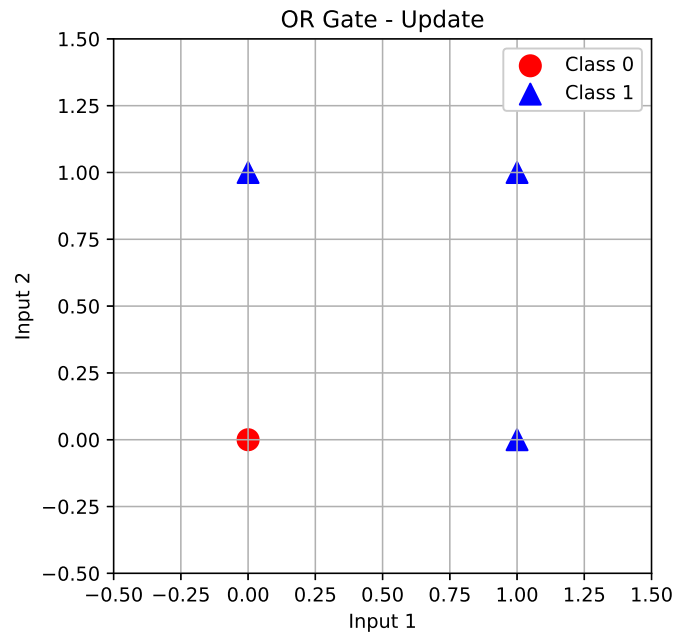


Figure 10: OR Gate - Weight Update 1

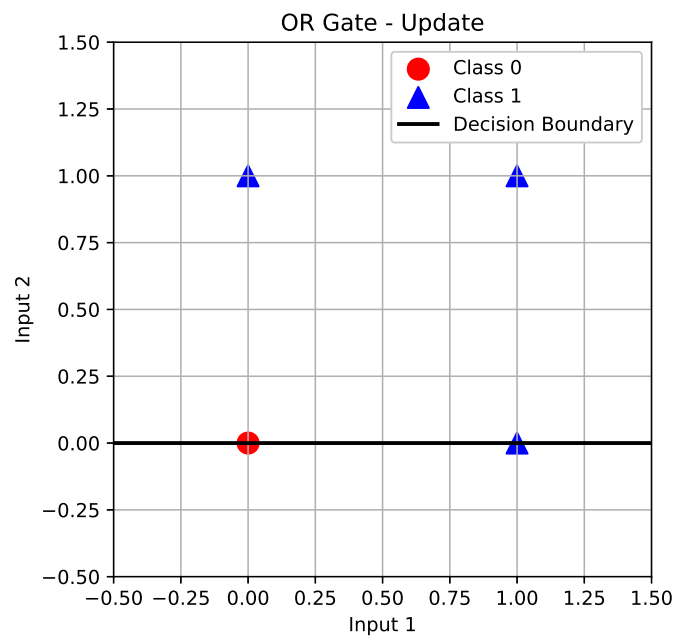


Figure 11: OR Gate - Weight Update 2

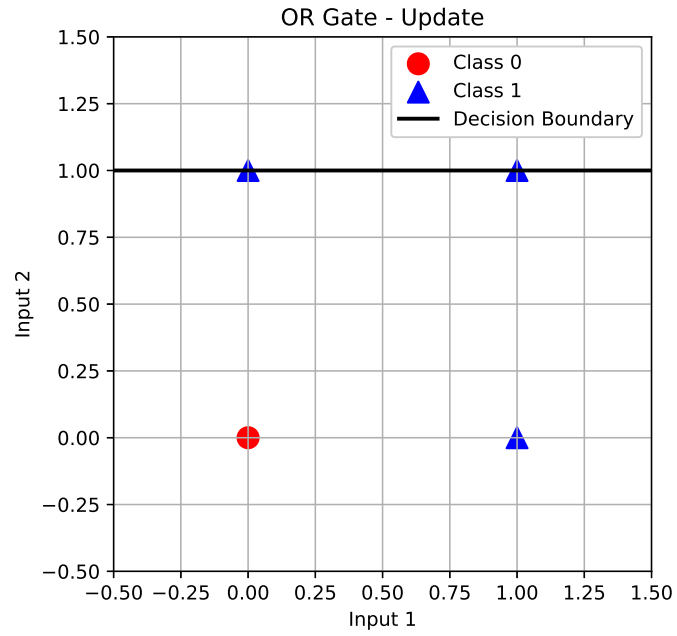


Figure 12: OR Gate - Weight Update 3

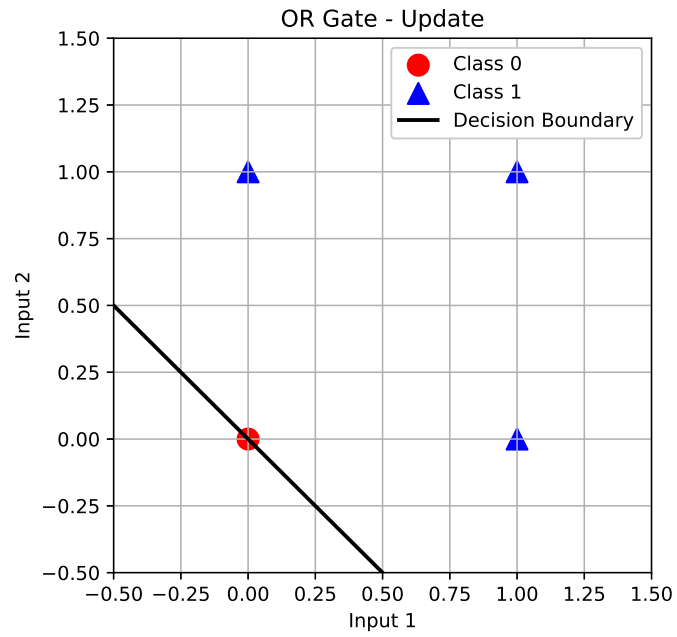


Figure 13: OR Gate - Weight Update 4

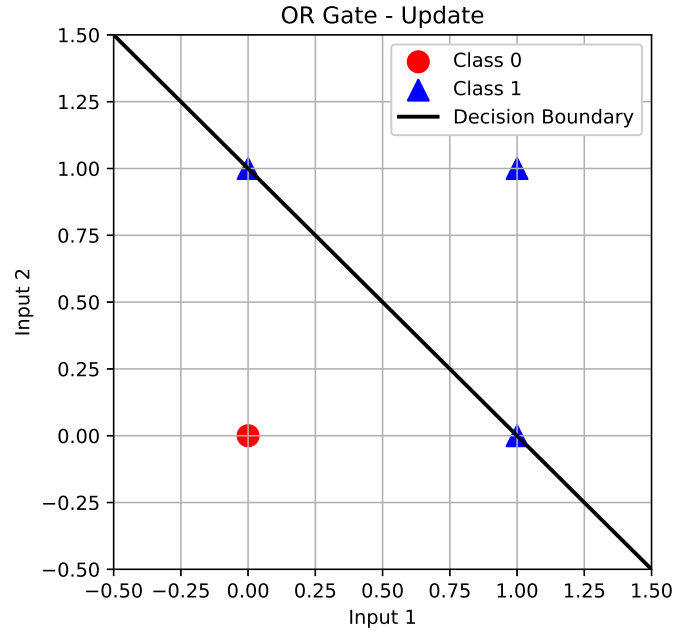


Figure 14: OR Gate - Weight Update 5

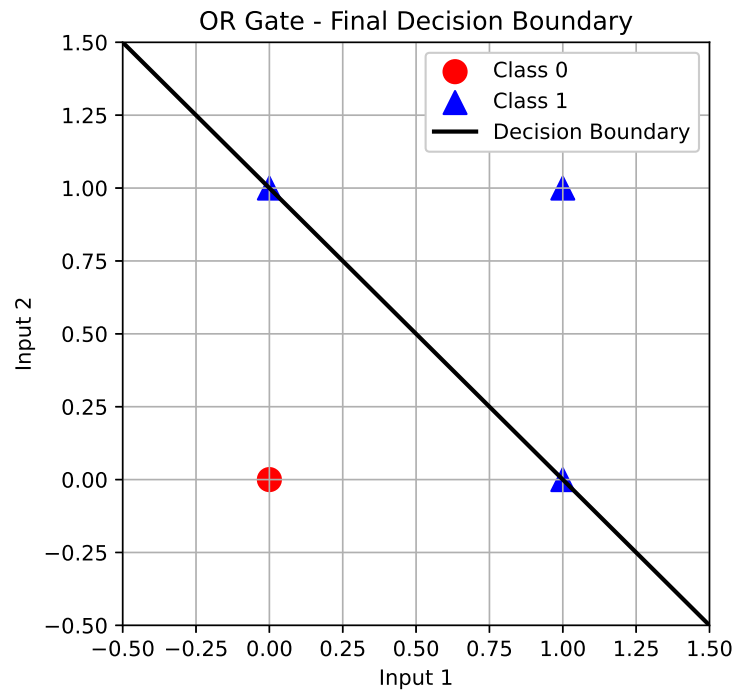


Figure 15: Final Decision Boundary for OR Gate

Observation:

The perceptron gradually updates its weights after each misclassified training sample. After a few updates the algorithm correctly classifies all input combinations of the OR gate. The final decision boundary clearly separates the positive and negative classes.

13.2 AND Gate

The AND gate produces an output of 1 only when both inputs are equal to 1.

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

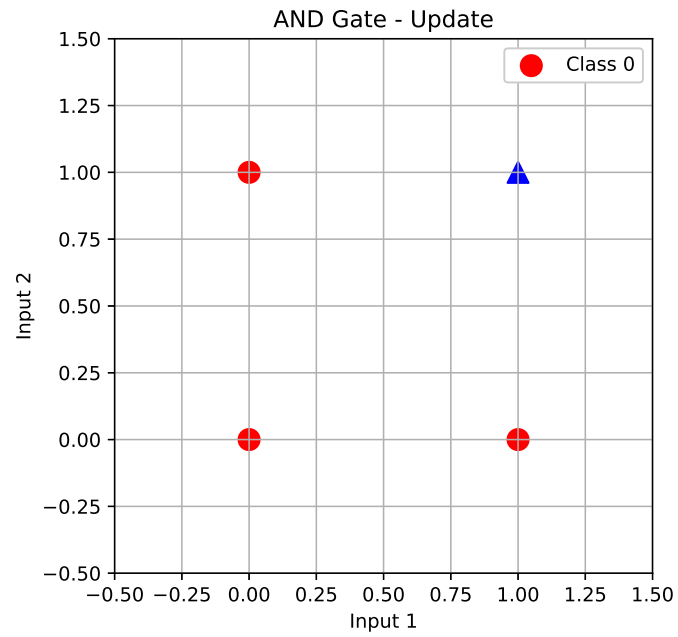


Figure 16: AND Gate - Weight Update 1

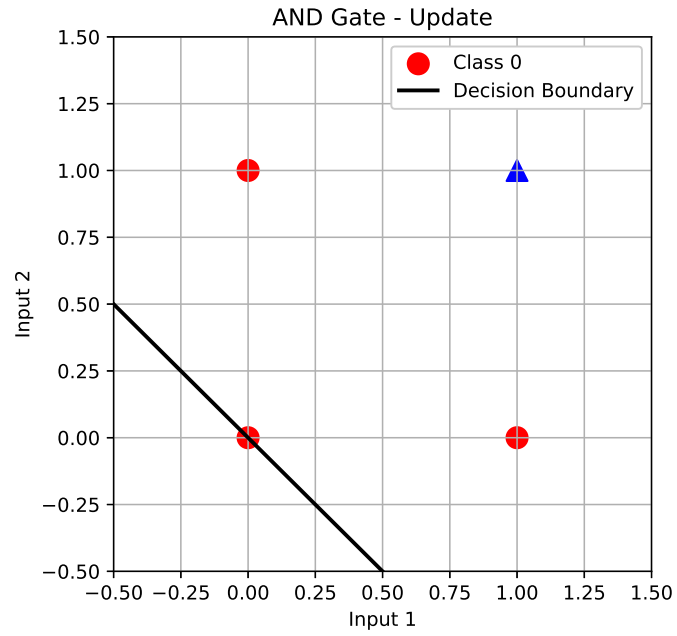


Figure 17: AND Gate - Weight Update 2

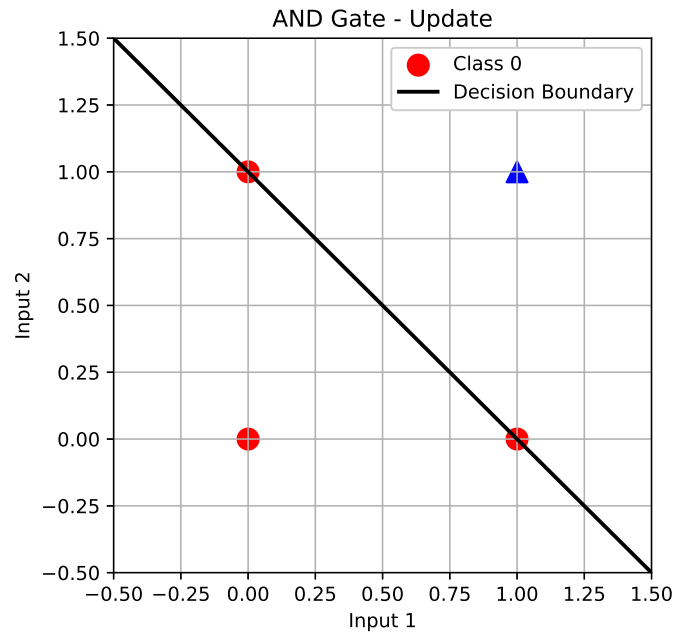


Figure 18: AND Gate - Weight Update 3

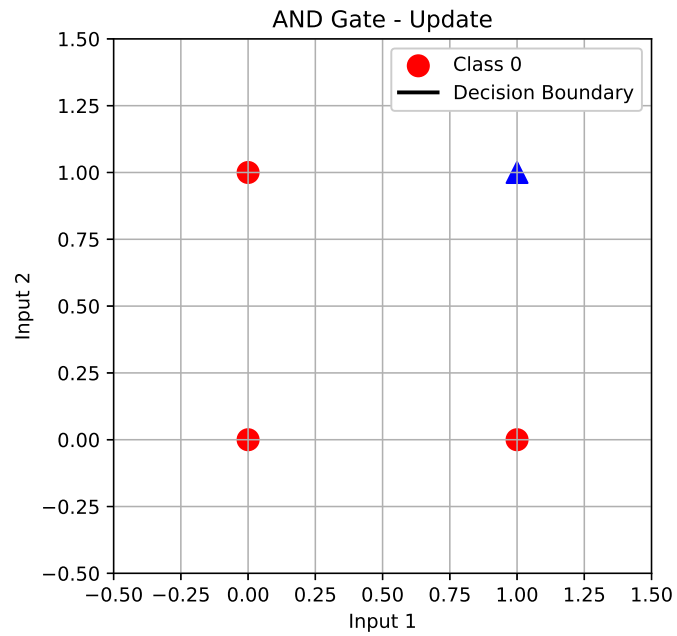


Figure 19: AND Gate - Weight Update 4

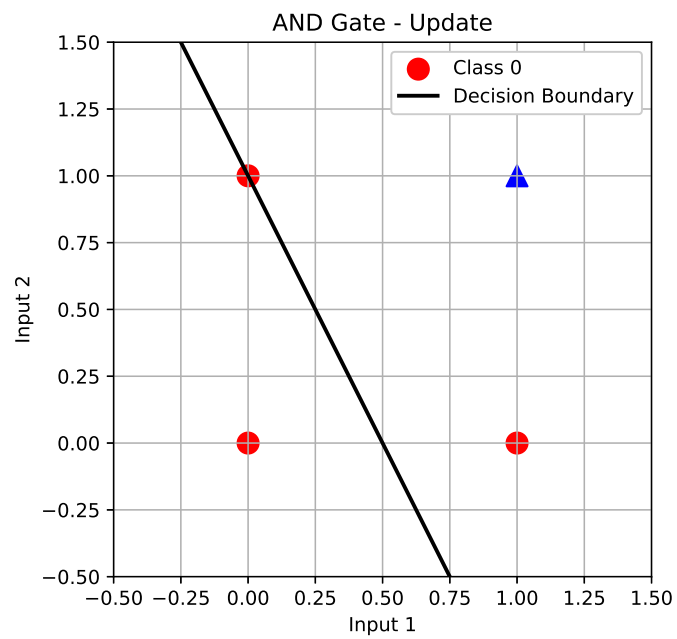


Figure 20: AND Gate - Weight Update 5

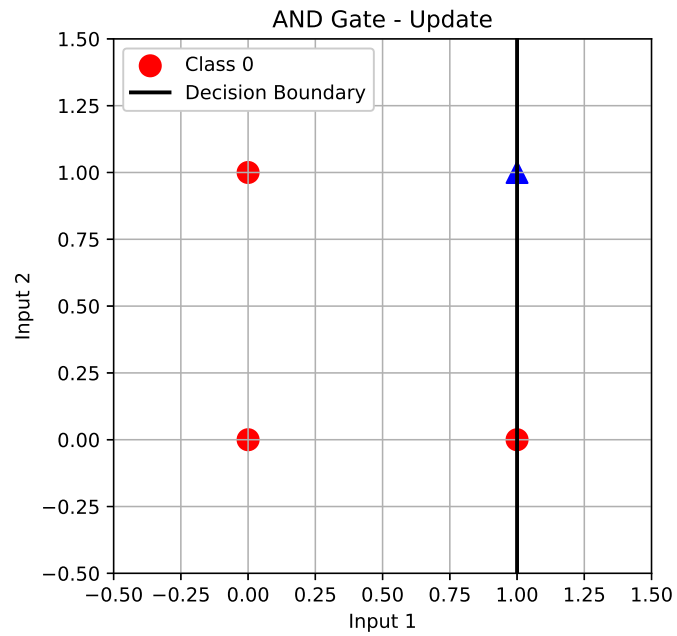


Figure 21: AND Gate - Weight Update 6

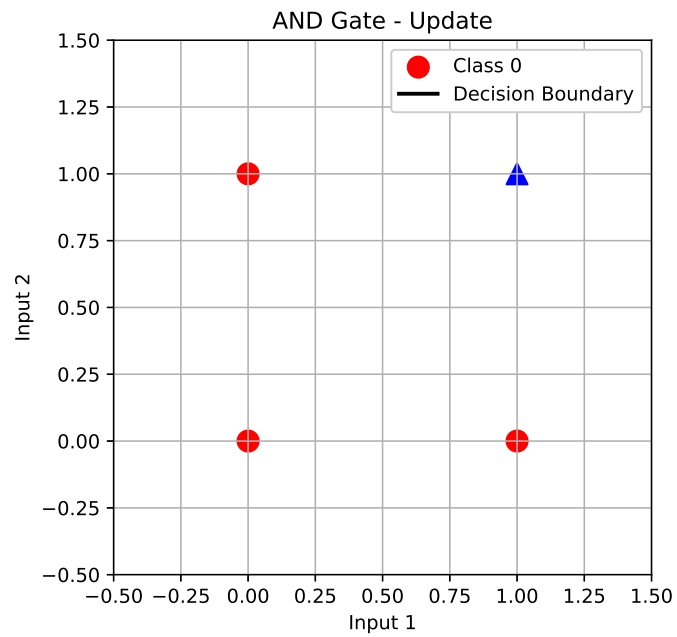


Figure 22: AND Gate - Weight Update 7

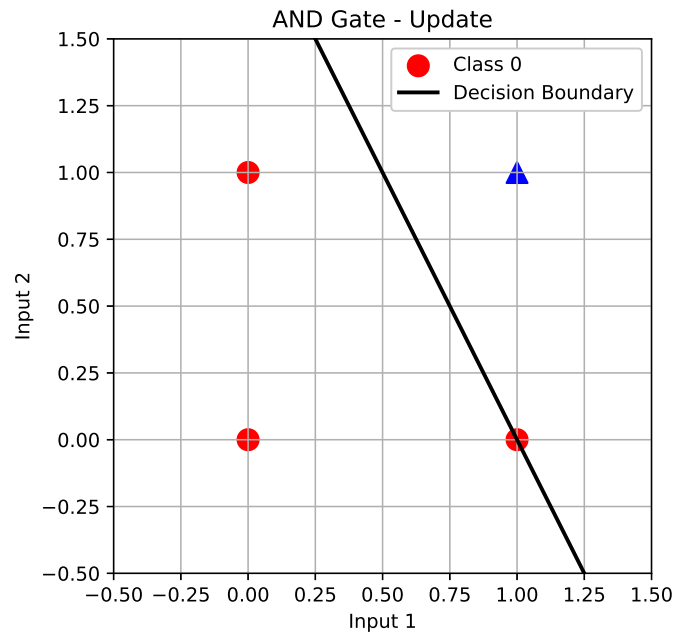


Figure 23: AND Gate - Weight Update 8

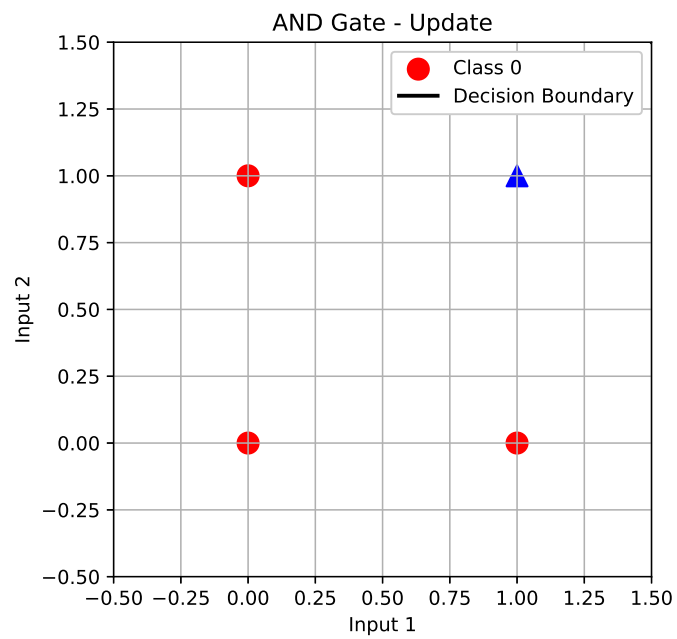


Figure 24: AND Gate - Weight Update 9

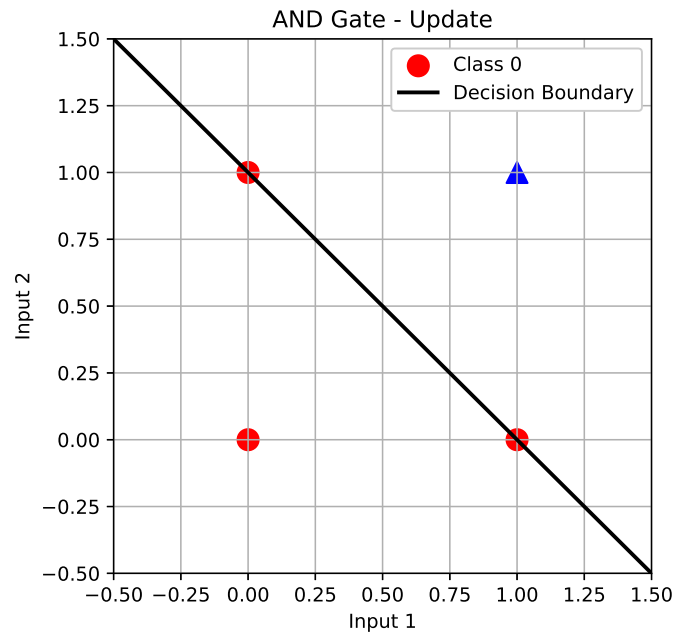


Figure 25: AND Gate - Weight Update 10

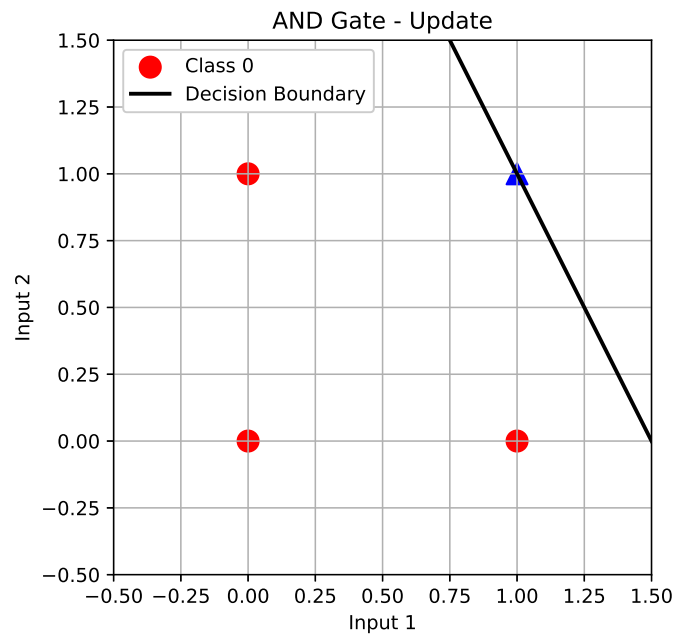


Figure 26: AND Gate - Weight Update 11

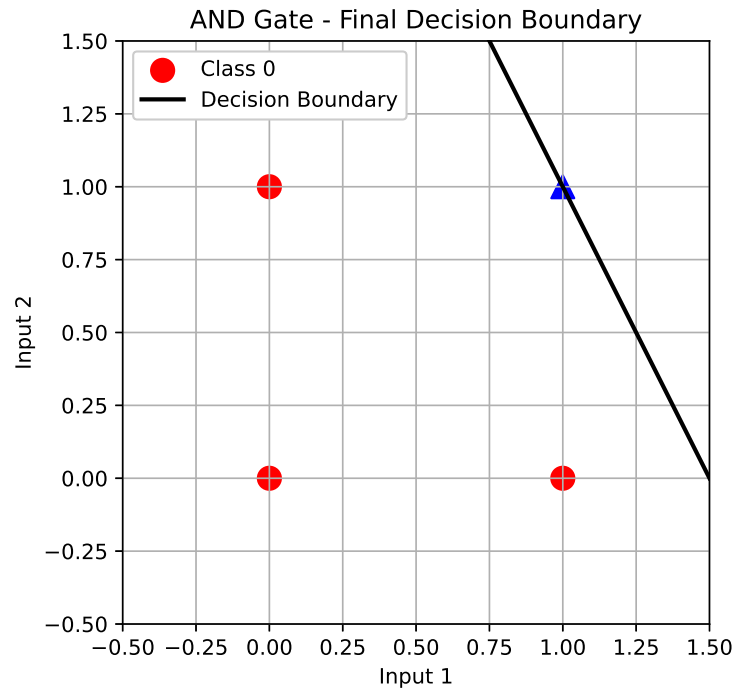


Figure 27: Final Decision Boundary for AND Gate

Observation:

The AND gate requires more weight updates. After many updates the perceptron finally learns a decision boundary that correctly classifies all four input combinations.

13.3 NOT Gate

The NOT gate is a single-input logic gate that reverses the input value.

Input	Output
0	1
1	0

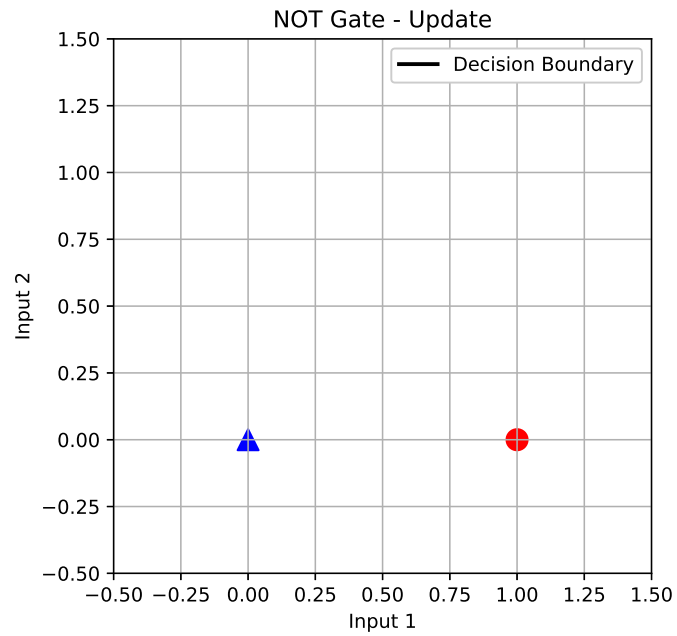


Figure 28: NOT Gate - First Weight Update

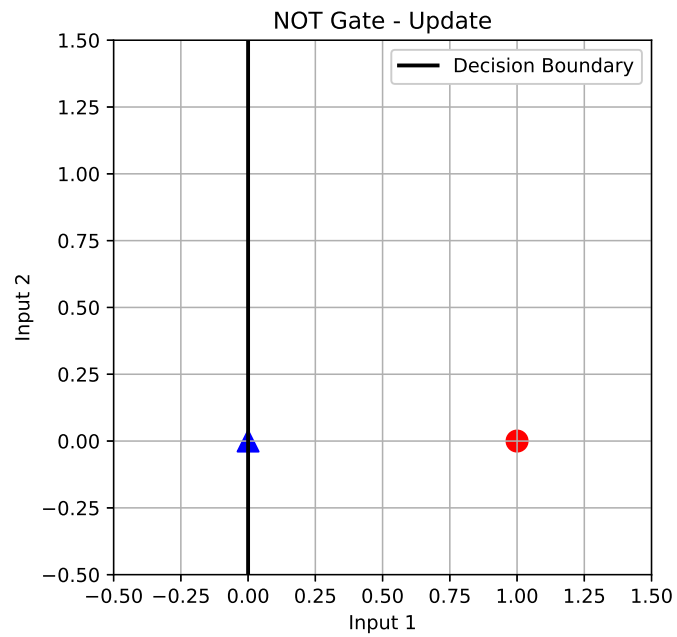


Figure 29: NOT Gate - Second Weight Update

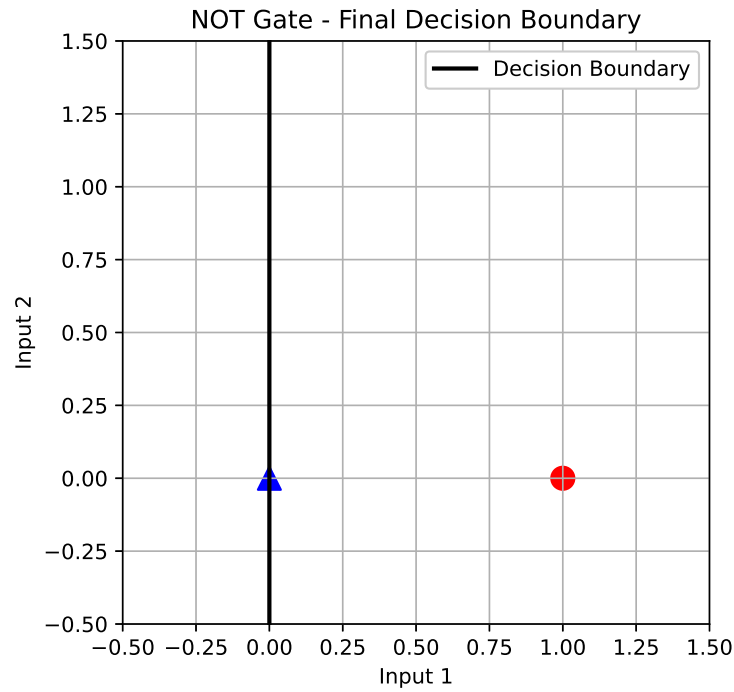


Figure 30: Final Decision Boundary for NOT Gate

Observation:

The NOT gate converges in only a few iterations because it contains only two training samples. The learned decision boundary correctly separates the two output classes.

13.4 Conclusion

The perceptron learning algorithm successfully learned the OR, AND, and NOT logic gates. The intermediate decision boundary plots show how the perceptron gradually adjusts its weights to achieve correct classification.